

Objetivo

Crear un PersistentVolume estático y vincularlo con un PersistentVolumeClaim. Desplegar MySQL utilizando un PVC como almacenamiento persistente. Verificar que los datos sobreviven a la eliminación y recreación del Pod. Crear un PVC mediante aprovisionamiento dinámico utilizando la StorageClass predeterminada de Minikube. Comparar de forma práctica el comportamiento de un volumen persistente con emptyDir.

Requisitos previos

- Haber completado las prácticas anteriores de Pods, Deployments, configuración, seguridad y networking.
- Tener Docker Desktop, Minikube y kubectl disponibles.
- Utilizar Visual Studio Code con Git Bash como terminal integrada.
- Conservar el clúster Minikube de tres nodos utilizado en las prácticas anteriores.
- Tener conocimientos básicos sobre sistemas de archivos y almacenamiento persistente.

Duración estimada

50 minutos

Introducción

En esta práctica recorrerás el ciclo de vida básico del almacenamiento persistente en Kubernetes. Crearás un PersistentVolume manual, solicitarás capacidad mediante un PersistentVolumeClaim, montarás ese almacenamiento en MySQL y demostrarás que los datos permanecen aunque el Pod sea eliminado y recreado. Después compararás este modelo con aprovisionamiento dinámico y con emptyDir para distinguir almacenamiento persistente y efímero.

Instrucciones

Preparación del directorio y namespace

En esta práctica crearás recursos de almacenamiento con alcance de clúster y recursos namespaced dentro de `lab7`.

Preparar el entorno

- **Paso 1.** Abre **Docker Desktop** y confirma que el motor continúa activo antes de verificar los nodos y el almacenamiento disponible.
- **Paso 2.** Abre **Visual Studio Code**, selecciona **Git Bash** como terminal integrada y crea el directorio correspondiente al laboratorio.

```
mkdir -p /c/LABS/kubernetes/lab7  
cd /c/LABS/kubernetes/lab7
```

 Copiar

- **Paso 3.** Comprueba que Minikube continúa activo y que los nodos del clúster permanecen en estado **Ready**.

```
minikube status  
kubectl get nodes
```

 Copiar

Salida esperada aproximada:

```
minikube
```

 Copiar

```
type: Control Plane  
host: Running  
kubelet: Running  
apiserver: Running
```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	...	...
minikube-m02	Ready	<none>	...	...
minikube-m03	Ready	<none>	...	...

Los nombres exactos, versión y antigüedad pueden variar, pero los tres nodos deben permanecer **Ready**.

- **Paso 4.** Crea el namespace **lab7** para aislar los PVC, Pods, Secret y demás recursos namespaced utilizados durante el ejercicio.

```
kubectl create namespace lab7
```

 Copiar

Salida esperada:

```
namespace/lab7 created
```

 Copiar

- **Paso 5.** Establece `lab7` como namespace predeterminado del contexto actual para reducir errores en comandos posteriores.

```
kubectl config set-context --current --  
namespace=lab7
```

 Copiar**Salida esperada aproximada:**

```
Context "<nombre-del-contexto>" modified.
```

 Copiar

- **Paso 6.** Consulta las StorageClasses existentes para identificar la opción predeterminada disponible en Minikube.

```
kubectl get storageclass
```

 Copiar**Salida esperada aproximada:**

NAME	PROVISIONER
RECLAIMPOLICY	VOLUMEBINDINGMODE
standard (default)	k8s.io/minikube-hostpath
Delete	Immediate

 Copiar

El nombre del provisioner y `VOLUMEBINDINGMODE` pueden variar entre versiones; debe existir una StorageClass marcada como `(default)` para la Tarea 3.

IMPORTANTE: No asumas el nombre de la StorageClass predeterminada hasta revisar esta salida. En instalaciones habituales de Minikube aparece `standard`, pero debes utilizar el valor real de tu clúster.



Tarea 1. Crear un PV estático y vincularlo con un PVC

En esta tarea crearás almacenamiento estático mediante `hostPath` y limitarás su uso al nodo `minikube` para evitar que el volumen apunte a directorios diferentes en un clúster multinodo.

Tarea 1.1. Preparar el directorio del nodo

- **Paso 7.** Crea el directorio que respaldará el PV dentro del nodo principal de Minikube.

```
minikube ssh -- "sudo mkdir -p /mnt/data/lab7-pv  
&& sudo chmod 777 /mnt/data/lab7-pv"
```

[Copiar](#)

Salida esperada:

El comando no debe mostrar errores. Puede finalizar sin imprimir texto.

- **Paso 8.** Comprueba que el directorio existe en el nodo antes de definir el PersistentVolume.

```
minikube ssh -- "ls -ld /mnt/data/lab7-pv"
```

[Copiar](#)

Salida esperada aproximada:

```
drwxrwxrwx ... /mnt/data/lab7-pv
```

[Copiar](#)

Tarea 1.2. Crear el PersistentVolume

- **Paso 9.** Crea el archivo `pv-static.yaml` para definir un PV de 1 GiB con política `Retain`.

```
touch pv-static.yaml
```

[Copiar](#)

- **Paso 10.** Agrega el manifiesto siguiente, incluyendo afinidad al nodo `minikube`.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-lab7-static
  labels:
    lab: lab7
    type: static
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: ""
  hostPath:
    path: /mnt/data/lab7-pv
    type: DirectoryOrCreate
  nodeAffinity:
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values:
                - minikube
```

 Copiar

- **Paso 11.** Valida y aplica el PV antes de crear cualquier reclamación de almacenamiento.

```
kubectl apply --dry-run=client -f pv-static.yaml
```

 Copiar

Salida esperada:

```
persistentvolume/pv-lab7-static created (dry run)
```

 Copiar

- **Paso 12.** Aplica el manifiesto validado para crear el PV de alcance de clúster.

```
kubectl apply -f pv-static.yaml
```

 Copiar

Salida esperada:

```
persistentvolume/pv-lab7-static created
```

 Copiar

- **Paso 13.** Comprueba que el PV se encuentra **Available** y todavía no está asociado con ningún PVC.

```
kubectl get pv pv-lab7-static
```

 Copiar**Salida esperada aproximada:**

NAME	CAPACITY	ACCESS MODES	RECLAIM
POLICY	STATUS	STORAGECLASS	
pv-lab7-static	1Gi	RWO	Retain
Available			

 Copiar**Tarea 1.3. Crear el PersistentVolumeClaim**

- **Paso 14.** Crea **pvc-static.yaml** para solicitar 500 MiB sobre el PV estático definido anteriormente.

```
touch pvc-static.yaml
```

 Copiar

- **Paso 15.** Agrega el manifiesto siguiente.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-lab7-static
  namespace: lab7
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
  storageClassName: ""
  volumeName: pv-lab7-static
```

 Copiar

- **Paso 16.** Valida el PVC antes de crearlo para confirmar que solicita explícitamente `pv-lab7-static` y no activa aprovisionamiento dinámico.

```
kubectl apply --dry-run=client -f pvc-
static.yaml
```

 Copiar

Salida esperada:

```
persistentvolumeclaim/pvc-lab7-static created (dry
run)
```

 Copiar

- **Paso 17.** Aplica el PVC y espera hasta que Kubernetes complete el binding con el PV estático.

```
kubectl apply -f pvc-static.yaml

kubectl wait --
for=jsonpath='{.status.phase}'=Bound pvc/pvc-
lab7-static -n lab7 --timeout=60s
```

 Copiar

Salida esperada:

```
persistentvolumeclaim/pvc-lab7-static created
persistentvolumeclaim/pvc-lab7-static condition met
```

[Copiar](#)

- **Paso 18.** Comprueba desde ambos recursos que el PVC quedó enlazado exactamente con `pv-lab7-static`.

```
kubectl get pvc pvc-lab7-static -n lab7
kubectl get pv pv-lab7-static
```

[Copiar](#)

Salida esperada aproximada:

NAME	STATUS	VOLUME
CAPACITY	ACCESS MODES	
pvc-lab7-static	Bound	pv-lab7-static 1Gi
RWO		

[Copiar](#)

El PV `pv-lab7-static` también debe aparecer con estado `Bound` y con `CLAIM` igual a `lab7/pvc-lab7-static`.

Tarea finalizada

Resultado esperado: PersistentVolume y PersistentVolumeClaim creados y vinculados correctamente, validando el estado `Bound` del almacenamiento estático.

Prompt de apoyo para explicar lo realizado

[↗ Explicar la Tarea 1 con IA](#)

Tarea 2. Desplegar MySQL utilizando almacenamiento persistente

En esta tarea montarás el PVC en `/var/lib/mysql`, crearás datos de prueba y demostrarás que esos datos sobreviven al ciclo de vida del Pod.

Tarea 2.1. Crear las credenciales

- **Paso 19.** Crea un Secret con credenciales exclusivamente destinadas al laboratorio.

```
kubectl create secret generic mysql-secret --  
from-literal=MYSQL_ROOT_PASSWORD='K8s-lab7-  
2026!' --from-literal=MYSQL_DATABASE=testdb -n  
lab7
```

 Copiar

Salida esperada:

```
secret/mysql-secret created
```

 Copiar

Tarea 2.2. Crear el Pod MySQL

- **Paso 20.** Crea el archivo `mysql-pvc.yaml` para montar `pvc-lab7-static` en la ruta de datos utilizada por MySQL.

```
touch mysql-pvc.yaml
```

 Copiar

- **Paso 21.** Agrega el manifiesto siguiente.

```
apiVersion: v1
kind: Pod
metadata:
  name: mysql-persistente
  namespace: lab7
  labels:
    app: mysql
spec:
  containers:
    - name: mysql
      image: mysql:8.0.42
      ports:
        - containerPort: 3306
      env:
        - name: MYSQL_ROOT_PASSWORD
          valueFrom:
            secretKeyRef:
              name: mysql-secret
              key: MYSQL_ROOT_PASSWORD
        - name: MYSQL_DATABASE
          valueFrom:
            secretKeyRef:
              name: mysql-secret
              key: MYSQL_DATABASE
      volumeMounts:
        - name: mysql-data
          mountPath: /var/lib/mysql
      readinessProbe:
        exec:
          command:
            - sh
            - -c
            - mysqladmin ping -h 127.0.0.1 -
              uroot -p"$MYSQL_ROOT_PASSWORD" --silent
          initialDelaySeconds: 10
          periodSeconds: 5
          timeoutSeconds: 3
          failureThreshold: 12
      resources:
        requests:
          cpu: 200m
          memory: 256Mi
        limits:
          cpu: 500m
```

 Copiar

```
memory: 512Mi
volumes:
- name: mysql-data
  persistentVolumeClaim:
    claimName: pvc-lab7-static
```

- **Paso 22.** Valida el manifiesto antes de crear el Pod, comprobando la referencia al Secret y al PVC sin modificar todavía el clúster.

```
kubectl apply --dry-run=client -f mysql-pvc.yaml
```

 Copiar

Salida esperada:

```
pod/mysql-persistente created (dry run)
```

 Copiar

- **Paso 23.** Aplica el Pod y espera hasta que la `readinessProbe` confirme que MySQL realmente acepta conexiones, no solo que el proceso arrancó.

```
kubectl apply -f mysql-pvc.yaml
```

 Copiar

```
kubectl wait --for=condition=Ready pod/mysql-
persistente -n lab7 --timeout=180s
```

Salida esperada:

```
pod/mysql-persistente created
pod/mysql-persistente condition met
```

 Copiar

- **Paso 24.** Comprueba en qué nodo fue programado el Pod y confirma que respeta la afinidad del volumen.

```
kubectl get pod mysql-persistente \
-n lab7 \
-o wide
```

 Copiar

Salida esperada aproximada:

NAME	READY	STATUS	IP
mysql-persistente	1/1	Running	10.244.x.x
minikube			

 Copiar

El Pod debe ejecutarse sobre `minikube` porque el PV está limitado mediante `nodeAffinity` a ese nodo.

Tarea 2.3. Crear datos persistentes

- **Paso 25.** Crea una tabla e inserta tres registros para disponer de información verificable antes de eliminar el Pod.

```
kubectl exec mysql-persistente \  
-n lab7 -- \  
mysql \  
-uroot \  
-p'K8s-lab7-2026!' \  
testdb \  
-e "  
    CREATE TABLE IF NOT EXISTS registros (  
        id INT AUTO_INCREMENT PRIMARY KEY,  
        mensaje VARCHAR(100)  
    );  
    INSERT INTO registros (mensaje) VALUES  
        ('Registro persistente 1'),  
        ('Registro persistente 2'),  
        ('Registro persistente 3');  
    SELECT * FROM registros;  
"
```

 Copiar**Salida esperada aproximada:**

id	mensaje
1	Registro persistente 1
2	Registro persistente 2
3	Registro persistente 3

 Copiar

Puede aparecer una advertencia de MySQL por utilizar la contraseña desde la línea de comandos; para este laboratorio es esperada.

- **Paso 26.** Obtén el número de registros y confirma que la base de datos contiene exactamente tres filas.

```
kubectl exec mysql-persistente \  
-n lab7 -- \  
mysql \  
-uroot \  
-p'K8s-lab7-2026!' \  
testdb \  
-sNe 'SELECT COUNT(*) FROM registros;'
```

 Copiar

Salida esperada:

```
3
```

 Copiar

Tarea 2.4. Eliminar y recrear el Pod

- **Paso 27.** Elimina únicamente el Pod y comprueba que el PVC continúa en estado `Bound`.

```
kubectl delete pod mysql-persistente -n lab7  
  
kubectl get pvc pvc-lab7-static -n lab7
```

 Copiar

Salida esperada aproximada:

```
pod "mysql-persistente" deleted
```

 Copiar

NAME	STATUS	VOLUME
pvc-lab7-static	Bound	pv-lab7-static

- **Paso 28.** Recrea el Pod utilizando exactamente el mismo manifiesto y espera nuevamente hasta que esté listo.

```
kubectl apply -f mysql-pvc.yaml
```

 Copiar

```
kubectl wait --for=condition=Ready pod/mysql-  
persistente -n lab7 --timeout=180s
```

Salida esperada:

```
pod/mysql-persistente created  
pod/mysql-persistente condition met
```

 Copiar

- **Paso 29.** Consulta la tabla después de recrear el Pod para demostrar que el contenido persistió fuera del ciclo de vida del contenedor.

```
kubectl exec mysql-persistente \  
-n lab7 -- \  
mysql \  
-uroot \  
-p'K8s-lab7-2026!' \  
testdb \  
-e 'SELECT * FROM registros;'
```

 Copiar

Salida esperada aproximada:

```
id  mensaje  
1   Registro persistente 1  
2   Registro persistente 2  
3   Registro persistente 3
```

 Copiar

Los tres registros deben conservarse porque `/var/lib/mysql` continúa respaldado por el mismo PV/PVC.

Concepto clave: El PVC no pertenece al ciclo de vida del Pod. El Pod puede eliminarse y recrearse mientras el volumen continúa conservando los datos almacenados.

Tarea finalizada

Resultado esperado: MySQL desplegado utilizando el PVC estático y datos conservados correctamente después de eliminar y recrear el Pod.

Prompt de apoyo para explicar lo realizado

➔ [Explicar la Tarea 2 con IA](#)

⚙️ Tarea 3. Utilizar aprovisionamiento dinámico

En esta tarea crearás un PVC sin definir manualmente un PV y dejarás que la StorageClass predeterminada del clúster aprovisione el almacenamiento.

Tarea 3.1. Identificar la StorageClass predeterminada

- **Paso 30.** Ejecuta el comando siguiente y localiza la StorageClass marcada como `(default)`.

```
kubectl get storageclass
```

 Copiar

- **Paso 31.** Guarda automáticamente el nombre de la StorageClass predeterminada para utilizarlo en las comprobaciones posteriores.

```
DEFAULT_SC=$(kubectl get storageclass \
  -o jsonpath='{range .items[?
  (@.metadata.annotations.storageclass\.kubernetes
  \.io/is-default-class=="true")]}{.metadata.name}
  {end}}')

echo "StorageClass predeterminada: $DEFAULT_SC"
```

 Copiar

Salida esperada aproximada:

```
StorageClass predeterminada: standard
```

 Copiar

El nombre puede ser diferente; la variable no debe quedar vacía.

IMPORTANTE: Si la variable queda vacía, revisa `kubectl get storageclass` y confirma que existe una clase predeterminada antes de continuar.

Tarea 3.2. Crear un PVC dinámico

- **Paso 32.** Crea `pvc-dynamic.yaml` sin especificar `storageClassName` ni `volumeName`, permitiendo que Kubernetes utilice la clase predeterminada.

```
touch pvc-dynamic.yaml
```

[Copiar](#)

- **Paso 33.** Agrega el manifiesto siguiente.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-lab7-dynamic
  namespace: lab7
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 200Mi
```

[Copiar](#)

- **Paso 34.** Valida y aplica el PVC sin especificar `storageClassName`, permitiendo que Kubernetes seleccione la StorageClass predeterminada.

```
kubectl apply --dry-run=client -f pvc-
dynamic.yaml
kubectl apply -f pvc-dynamic.yaml
```

[Copiar](#)

Salida esperada:

```
persistentvolumeclaim/pvc-lab7-dynamic created (dry
run)
persistentvolumeclaim/pvc-lab7-dynamic created
```

 Copiar

- **Paso 35.** Consulta el estado del PVC y la modalidad de binding de la StorageClass para interpretar correctamente `Bound` o `Pending`.

```
kubectl get pvc pvc-lab7-dynamic -n lab7

kubectl get storageclass "$DEFAULT_SC" -o
jsonpath='{.volumeBindingMode}{"\n"}'
```

 Copiar

Salida esperada aproximada:

```
NAME              STATUS    VOLUME
pvc-lab7-dynamic  Bound    pvc-...
Immediate
```

 Copiar

Si la StorageClass muestra `WaitForFirstConsumer`, es válido que el PVC continúe `Pending` hasta crear `nginx-dynamic` en la Tarea 3.3.

- **Paso 36.** Consulta el nombre del PV creado por el provisioner y revisa la StorageClass utilizada.

```
DYNAMIC_PV=$(kubectl get pvc pvc-lab7-dynamic \
-n lab7 \
-o jsonpath='{.spec.volumeName}')

echo "PV dinámico: $DYNAMIC_PV"

kubectl get pv "$DYNAMIC_PV"
```

 Copiar

Tarea 3.3. Consumir el PVC dinámico

- **Paso 37.** Crea el archivo `pod-dynamic.yaml` para montar el PVC dinámico dentro de un Pod NGINX.

```
touch pod-dynamic.yaml
```

 Copiar

- **Paso 38.** Agrega el manifiesto siguiente.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-dynamic
  namespace: lab7
spec:
  containers:
    - name: nginx
      image: nginx:1.27-alpine
      volumeMounts:
        - name: web-data
          mountPath: /usr/share/nginx/html
  volumes:
    - name: web-data
      persistentVolumeClaim:
        claimName: pvc-lab7-dynamic
```

 Copiar

- **Paso 39.** Aplica el Pod, espera a que esté listo y escribe un archivo dentro del volumen aprovisionado dinámicamente.

```
kubectl apply -f pod-dynamic.yaml
```

 Copiar

```
kubectl wait \
  --for=condition=Ready \
  pod/nginx-dynamic \
  -n lab7 \
  --timeout=120s

kubectl exec nginx-dynamic \
  -n lab7 -- \
  sh -c 'echo "<h1>PVC dinamico operativo</h1>"
> /usr/share/nginx/html/index.html'
```

- **Paso 40.** Lee nuevamente el archivo para confirmar que el volumen se encuentra montado y disponible.

```
kubectl exec nginx-dynamic -n lab7 -- sh -c 'cat /usr/share/nginx/html/index.html'
```

 Copiar

Salida esperada:

```
<h1>PVC dinamico operativo</h1>
```

 Copiar

Tarea finalizada

Resultado esperado: PVC dinámico aprovisionado mediante la StorageClass predeterminada y consumido correctamente desde un Pod NGINX.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 3 con IA](#)

Tarea 4. Comparar emptyDir y PVC

En esta tarea crearás un volumen efímero y demostrarás que sus datos desaparecen cuando el Pod deja de existir.

Tarea 4.1. Crear el Pod con emptyDir

- **Paso 41.** Crea el archivo `pod-emptydir.yaml` con un contenedor que permanezca en ejecución sin generar automáticamente archivos dentro del volumen.

```
touch pod-emptydir.yaml
```

 Copiar

- **Paso 42.** Agrega el manifiesto siguiente.

```
apiVersion: v1
kind: Pod
metadata:
  name: emptydir-demo
  namespace: lab7
spec:
  containers:
    - name: app
      image: busybox:1.36
      command:
        - sh
        - -c
        - sleep 3600
      volumeMounts:
        - name: temp-data
          mountPath: /datos
  volumes:
    - name: temp-data
      emptyDir: {}
```

 Copiar

- **Paso 43.** Aplica el Pod y espera hasta que se encuentre listo antes de escribir datos.

```
kubectl apply --dry-run=client -f pod-
emptydir.yaml
kubectl apply -f pod-emptydir.yaml

kubectl wait --for=condition=Ready pod/emptydir-
demo -n lab7 --timeout=60s
```

 Copiar

Salida esperada:

```
pod/emptydir-demo created (dry run)
pod/emptydir-demo created
pod/emptydir-demo condition met
```

 Copiar

Tarea 4.2. Escribir y perder el dato

- **Paso 44.** Crea manualmente un archivo dentro de `emptyDir` y comprueba que existe durante la vida actual del Pod.

```
kubectl exec emptydir-demo \  
-n lab7 -- \  
sh -c 'echo "Dato temporal lab7" >  
/datos/temporal.txt'  
  
kubectl exec emptydir-demo -n lab7 -- sh -c 'cat  
/datos/temporal.txt'
```

 Copiar

Salida esperada:

```
Dato temporal lab7
```

 Copiar

- **Paso 45.** Elimina y recrea el Pod utilizando el mismo manifiesto.

```
kubectl delete pod emptydir-demo -n lab7  
  
kubectl apply -f pod-emptydir.yaml  
  
kubectl wait --for=condition=Ready pod/emptydir-  
demo -n lab7 --timeout=60s
```

 Copiar

Salida esperada:

```
pod "emptydir-demo" deleted  
pod/emptydir-demo created  
pod/emptydir-demo condition met
```

 Copiar

- **Paso 46.** Intenta leer el archivo anterior y confirma que ya no existe.

```
kubectl exec emptydir-demo \  
-n lab7 -- \  
sh -c 'test -f /datos/temporal.txt && cat  
/datos/temporal.txt || echo "Archivo no  
encontrado"'
```

 Copiar

Salida esperada:

Archivo no encontrado

 Copiar

Concepto clave: `emptyDir` vive mientras existe el Pod. Un contenedor puede reiniciarse y conservar el volumen dentro del mismo Pod, pero al eliminar el Pod se elimina también `emptyDir`.

Tarea finalizada

Resultado esperado: Diferencia entre `emptyDir` y PVC comprobada mediante la pérdida de datos efímeros y conservación del almacenamiento persistente.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 4 con IA](#)

✓ Tarea 5. Validar almacenamiento persistente

En esta tarea comprobarás los estados esenciales del almacenamiento utilizado durante la práctica.

Tarea 5.1. Revisar PV y PVC

- **Paso 47.** Consulta los PV y PVC creados para identificar claramente almacenamiento estático y dinámico.

```
kubectl get pv
kubectl get pvc -n lab7
```

 Copiar

Salida esperada aproximada:

PV:

```
pv-lab7-static    1Gi      RWO    Retain    Bound
lab7/pvc-lab7-static
pvc-...          ...      RWO    Delete    Bound
lab7/pvc-lab7-dynamic
```

 Copiar

PVC:

```
pvc-lab7-static    Bound    pv-lab7-static
pvc-lab7-dynamic   Bound    pvc-...
```

El nombre del PV dinámico es generado por Kubernetes y será diferente en cada ejecución.

Tarea 5.2. Ejecutar validación final

- **Paso 48.** Ejecuta el bloque siguiente para validar binding estático, ubicación de MySQL, persistencia, aprovisionamiento dinámico y pérdida de datos de `emptyDir`.

```
echo "=== Validacion final de la Practica 7 ==="
```

[Copiar](#)

```
STATIC_PVC=$(kubectl get pvc pvc-lab7-static \
-n lab7 \
-o jsonpath='{.status.phase}')

[ "$STATIC_PVC" = "Bound" ] \
&& echo "✅ PVC estatico Bound" \
|| echo "❌ PVC estatico: $STATIC_PVC"

STATIC_PV=$(kubectl get pv pv-lab7-static \
-o jsonpath='{.status.phase}')

[ "$STATIC_PV" = "Bound" ] \
&& echo "✅ PV estatico Bound" \
|| echo "❌ PV estatico: $STATIC_PV"

MYSQL_NODE=$(kubectl get pod mysql-persistente \
-n lab7 \
-o jsonpath='{.spec.nodeName}')

[ "$MYSQL_NODE" = "minikube" ] \
&& echo "✅ MySQL programado en minikube" \
|| echo "❌ MySQL programado en: $MYSQL_NODE"

MYSQL_COUNT=$(kubectl exec mysql-persistente \
-n lab7 -- \
mysql \
-u root \
-p 'K8s-lab7-2026!' \
testdb \
-sNe 'SELECT COUNT(*) FROM registros;' \
2>/dev/null)

[ "$MYSQL_COUNT" = "3" ] \
&& echo "✅ Datos MySQL persistentes: 3 registros" \
|| echo "❌ Registros encontrados: $MYSQL_COUNT"

DYNAMIC_PVC=$(kubectl get pvc pvc-lab7-dynamic \
-n lab7 \
-o jsonpath='{.status.phase}')
```

```
[ "$DYNAMIC_PVC" = "Bound" ] \
&& echo "✅ PVC dinamico Bound" \
|| echo "❌ PVC dinamico: $DYNAMIC_PVC"

DYNAMIC_FILE=$(kubectl exec nginx-dynamic \
-n lab7 -- \
sh -c 'cat /usr/share/nginx/html/index.html' \
2>/dev/null)

echo "$DYNAMIC_FILE" | grep -q "PVC dinamico
operativo" \
&& echo "✅ PVC dinamico montado y con datos" \
|| echo "❌ No se pudo validar el contenido
del PVC dinamico"

EMPTY_FILE=$(kubectl exec emptydir-demo \
-n lab7 -- \
sh -c 'test -f /datos/temporal.txt && echo
existe || echo ausente')

[ "$EMPTY_FILE" = "ausente" ] \
&& echo "✅ emptyDir perdio el dato al recrear
el Pod" \
|| echo "❌ El archivo temporal sigue
presente"

echo "=== Fin de validacion ==="
```

Salida esperada:

```
=== Validacion final de la Practica 7 ===
✅ PVC estatico Bound
✅ PV estatico Bound
✅ MySQL programado en minikube
✅ Datos MySQL persistentes: 3 registros
✅ PVC dinamico Bound
✅ PVC dinamico montado y con datos
✅ emptyDir perdio el dato al recrear el Pod
=== Fin de validacion ===
```

 Copiar

IMPORTANTE: No elimines el clúster Minikube. Puedes conservar los recursos de `lab7` hasta terminar la revisión de esta práctica.

Tarea finalizada

Resultado esperado: Recursos de almacenamiento validados correctamente, confirmando PV, PVC, persistencia de MySQL y aprovisionamiento dinámico.

Prompt de apoyo para explicar lo realizado

➔ [Explicar la Tarea 5 con IA](#)

🔧 Resolución de problemas

Problema 1. El PVC estático permanece Pending

Síntoma: `pvc-lab7-static` no alcanza estado `Bound`.

Causa probable: El PV no existe, está ocupado, no coincide con el nombre solicitado o su capacidad y access mode no satisfacen el PVC.

Solución:

```
kubectl get pv pv-lab7-static
kubectl describe pv pv-lab7-static
kubectl describe pvc pvc-lab7-static -n lab7
```

 Copiar

Comprueba que `volumeName` sea `pv-lab7-static`, que el PV esté disponible y que `storageClassName` sea vacío en ambos recursos.

Problema 2. mysql-persistente permanece Pending

Síntoma: El PVC está `Bound`, pero el Pod MySQL no se programa.

Causa probable: El PV está limitado mediante `nodeAffinity` al nodo `minikube` y ese nodo no está disponible o no dispone de recursos suficientes.

Solución:

```
kubectl describe pod mysql-persistente -n lab7
kubectl get node minikube
kubectl describe node minikube
```

 Copiar

Confirma que el nodo `minikube` esté `Ready` antes de modificar el volumen.

Problema 3. MySQL entra en CrashLoopBackOff

Síntoma: El Pod se programa pero el contenedor reinicia repetidamente.

Causa probable: MySQL encuentra datos incompatibles en el volumen o existe un problema con las variables obtenidas desde el Secret.

Solución:

```
kubectl logs mysql-persistente -n lab7
kubectl describe pod mysql-persistente -n lab7
kubectl get secret mysql-secret -n lab7
```

 Copiar

No elimines el contenido del volumen hasta identificar la causa en los logs.

Problema 4. El PVC dinámico permanece Pending

Síntoma: `pvc-lab7-dynamic` no obtiene un PV.

Causa probable: No existe una StorageClass predeterminada o el provisioner asociado no está disponible.

Solución:

```
kubectl get storageclass
minikube addons list | grep storage
kubectl describe pvc pvc-lab7-dynamic -n lab7
```

 Copiar

Si Minikube no tiene habilitado el provisioner predeterminado, revisa los addons de almacenamiento disponibles antes de cambiar el manifiesto.

Problema 5. Los datos de MySQL desaparecieron después de recrear el Pod

Síntoma: La tabla o registros creados anteriormente ya no existen.

Causa probable: El Pod se recreó utilizando otro PVC, el volumen fue eliminado o el PV estático fue modificado.

Solución:

```
kubectl get pod mysql-persistente \
  -n lab7 \
  -o
jsonpath='{.spec.volumes[0].persistentVolumeClaim.claimName}'

kubectl get pvc pvc-lab7-static -n lab7
kubectl get pv pv-lab7-static
```

[Copiar](#)

El Pod debe continuar utilizando `pvc-lab7-static`, y el PVC debe permanecer asociado con `pv-lab7-static`.

Problema 6. Git Bash transforma rutas Linux dentro del contenedor

Síntoma: Un comando que intenta acceder a `/usr/share/nginx/html` o `/datos` termina utilizando una ruta similar a `C:/Program Files/Git/...`.

Causa probable: Git Bash/MSYS2 convierte automáticamente argumentos que comienzan con `/` antes de entregarlos a `kubectl`.

Solución:

Ejecuta la operación dentro de `sh -c`.

```
kubectl exec nginx-dynamic \
  -n lab7 -- \
  sh -c 'cat /usr/share/nginx/html/index.html'
```

[Copiar](#)

El mismo patrón debe utilizarse para rutas como `/datos/temporal.txt`.

Problema 7. MySQL está Running pero todavía no acepta consultas

Síntoma: El Pod aparece `Running`, pero `mysql` devuelve un error de conexión durante los primeros segundos.

Causa probable: El proceso principal ya inició, pero MySQL todavía está inicializando el directorio de datos o las tablas internas.

Solución:

La práctica utiliza una `readinessProbe` con `mysqladmin ping`. Espera a que Kubernetes marque el Pod como `Ready`.

```
kubectl wait \
  --for=condition=Ready \
  pod/mysql-persistente \
  -n lab7 \
  --timeout=180s
```

[Copiar](#)

Si no alcanza `Ready`, revisa:

```
kubectl logs mysql-persistente -n lab7
kubectl describe pod mysql-persistente -n lab7
```

[Copiar](#)

Problema 8. El PVC dinámico está Pending antes de crear nginx-dynamic

Síntoma: El PVC dinámico no obtiene inmediatamente un PV aunque exista una StorageClass predeterminada.

Causa probable: La StorageClass utiliza `WaitForFirstConsumer`, por lo que el provisioner espera a conocer dónde se programará el primer Pod consumidor.

Solución:

Comprueba la modalidad:

```
kubectl get storageclass "$DEFAULT_SC" \
  -o jsonpath='{.volumeBindingMode}{ "\n"}'
```

[Copiar](#)

Si devuelve:

```
WaitForFirstConsumer
```

[Copiar](#)

continúa con la creación de `nginx-dynamic`. Después verifica nuevamente el PVC; debe pasar a `Bound`.

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar la práctica habrás creado y consumido almacenamiento persistente mediante PV y PVC, verificado la conservación de datos después de recrear un Pod y utilizado aprovisionamiento dinámico mediante StorageClass.

Notas y consideraciones

- Esta práctica utiliza el namespace `lab7`.
- El PV estático usa `hostPath` únicamente con fines de laboratorio y queda asociado al nodo minikube mediante `nodeAffinity`.
- En producción se utilizan normalmente soluciones CSI y almacenamiento proporcionado por infraestructura externa.
- No elimines el clúster Minikube durante la práctica.
- En Git Bash, las rutas Linux utilizadas dentro de contenedores se ejecutan mediante `sh -c` para evitar la conversión automática de rutas de `MSYS2`.
- El PV estático utiliza `storageClassName ""` para impedir que la StorageClass predeterminada intervenga en el binding manual.
- La política `Retain` conserva los datos del PV aunque posteriormente se elimine el PVC; no elimina automáticamente el contenido de `hostPath`.

URL de referencia

- [Persistent Volumes](#)
- [Storage Classes](#)
- [Volumes](#)
- [Minikube Persistent Volumes](#)